

Jetson ↔ Mega 통신 프로토콜 변경 안내

제어 권한 이전(Jetson → Mega) — 2026-07-15 / STREW Jetson Robot 프로젝트

1. 이 문서의 목적

외부 팀 회의 결과로 로봇의 REPLACE/OBSERVE/SKIP 판단 권한이 Jetson에서 Mega로 이전되기로 결정되었습니다. 이 문서는 (1) 무엇이 왜 바뀌었는지, (2) 전체 구조가 어떻게 달라졌는지, (3) Jetson↔Mega 사이에 실제로 오갈 메시지의 정확한 필드값과 예시, (4) Mega 펌웨어가 새로 구현해야 할 처리 로직을 한 문서에 정리해 전달하기 위해 작성되었습니다.

2. 변경 배경과 근거

기존 구조에서는 Jetson이 카메라/AI로 식물 상태를 판단하고, 그 판단(REPLACE/OBSERVE/SKIP)을 그대로 명령으로 만들어 Mega에 내려보냈습니다. Mega는 순수하게 그 명령을 받아 물리적으로 수행하기만 했습니다. 이번 변경은 이 결정 로직 자체를 Mega로 옮기는 것으로, 다음과 같은 이유로 결정되었습니다.

- Jetson의 연산 부하 감소 — 결정 로직 자체는 가벼운 맵핑 규칙이라 Mega(MCU)에서도 충분히 처리 가능합니다.
- 현장 대응성 — Mega가 물리 동작 도중 즉시 판단하고 대응할 수 있어, 매 단계마다 Jetson의 응답을 기다릴 필요가 줄어듭니다.
- 역할 분담 재정비 — Jetson은 "AWS 게이트웨이 + 비전 판단 제공자"로, Mega는 "판단 + 실행"으로 책임이 재편됩니다.

주의: 이 변경으로 기존 설계 원칙 하나가 명시적으로 폐기됩니다 — 예전에는 "Mega는 명령을 받기 전엔 절대 스스로 움직이지 않으며, Jetson의 판단은 항상 Mega에 명령을 보내기 이전에 끝나 있어야 한다"는 원칙이 있었습니다. 새 구조에서는 Mega가 동작 도중에 먼저 Jetson에 말을 걸 수 있고(비전 확인 요청), 그 결과를 받은 뒤 스스로 판단합니다. 통신 방향이 사실상 양방향으로 바뀌었다는 점이 이번 변경의 핵심입니다.

3. 이전 구조 vs 신규 구조

이전 구조 (Jetson이 결정)

Jetson: 카메라로 상태 판단 → REPLACE/OBSERVE/SKIP 결정 → Mega에 명령(command+target) 전송 → Mega가 단계별 진행상황(RECEIVED→...→COMPLETE)을 스트리밍으로 응답 → Jetson이 AWS에 실시간 중계 → (REPLACE였다면) Jetson이 완료 후 재확인 비전 스캔까지 수행.

신규 구조 (Mega가 결정)

Jetson: AWS(또는 로컬)에서 작업을 받아 Mega에 "이 셀로 가라"(대상 셀)만 지정 → 이후 응답을 기다리지 않음.

Mega: 셀로 이동 등 내부 동작 수행 → 필요할 때마다(1회 이상) Jetson에 "지금 비전 상태 확인해줘"

요청 → Jetson이 카메라/AI로 판단한 날것 상태(healthy 등)만 회신 → Mega가 그 값으로 REPLACE/OBSERVE/SKIP을 스스로 판단하고 물리 동작 수행 → 필요하면 완료 후 검증을 위해 같은 방식으로 비전을 한 번 더 요청할 수 있음(별도의 특수 메커니즘이 아니라 동일한 요청/응답을 재사용) → 최종적으로 결과를 Jetson에 1회 보고.
 Jetson: 그 보고를 그대로 AWS에 전달(중계)만 함 — 내용을 해석하거나 재판단하지 않음.

4. 역할 분담 원칙 (신규)

구분	이전	신규
무엇을 할지(WHAT) 결정	Jetson	Mega
비전 상태 판단(status 계산)	Jetson (그대로 유지)	Jetson (그대로 유지 — 카메라가 물리적으로 Jetson에 있음)
통신 개시 주체	Jetson만 (단방향)	Jetson과 Mega 모두 가능 (양방향)
물리 제어(어떻게 할지, HOW)	Mega	Mega (변경 없음)
AWS 보고	Jetson	Jetson (변경 없음 — Mega는 인터넷 연결이 없으므로 Jetson이 계속 게이트웨이 역할)

5. 신규 메시지 프로토콜 상세

전송 형식은 기존과 동일합니다 — 한 줄 JSON + 개행문자(\n), UTF-8 인코딩. 모든 신규 메시지는 "type" 필드로 종류를 구분합니다(기존 방식의 "command"/"task" 필드와는 별개의 새 규칙입니다). 총 4종의 메시지가 있습니다.

type 값	방향	필드	설명
ASSIGN_TARGET	Jetson → Mega	target (문자열, 예: "cell_5")	이번 사이클에 처리할 셀만 지정합니다. 판단(REPLACE 등)은 포함하지 않습니다. 한 사이클당 1회 전송됩니다.
REQUEST_VISION	Mega → Jetson	(없음)	Mega가 동작 도중 아무 때나, 필요한 만큼 여러 번 보낼 수 있습니다. (예: 셀 도착 직후 1회, REPLACE 완료 후 검증용으로 1회 더 등)
VISION_RESULT	Jetson → Mega	status (문자열: healthy / powdery_mildew / missing_plant / empty_cell)	REQUEST_VISION에 대한 응답입니다. 판단 없이 날것의 비전 결과만 답습니다. 이 값으로 무엇을 할지는 전적으로 Mega가 결정합니다.
REPORT_RESULT	Mega → Jetson	target, execute_task (Mega가 내부에서 정한 REPLACE/OBSERVE/SKIP), completion, success (선택, boolean)	한 사이클이 끝날 때 1회 전송합니다. Jetson은 이 내용을 그대로 AWS에 중계만 합니다.

메시지 예시 (JSON)

ASSIGN_TARGET (Jetson → Mega):

```
{"type": "ASSIGN_TARGET", "target": "cell_5"}
```

REQUEST_VISION (Mega → Jetson):

```
{"type": "REQUEST_VISION"}
```

VISION_RESULT (Jetson → Mega):

```
{"type": "VISION_RESULT", "status": "powdery_mildew"}
```

REPORT_RESULT (Mega → Jetson):

```
{"type": "REPORT_RESULT", "target": "cell_5", "execute_task": "REPLACE", "completion": "COMPLETE", "success": true}
```

인코딩 규칙: `json.dumps(payload, separators=(",", ":"), ensure_ascii=False)` + "\n" 을 UTF-8로 인코딩한 바이트를 그대로 시리얼로 전송합니다. Mega 쪽에서는 개행문자(\n)가 나올 때까지 한 줄을 읽어 UTF-8로 디코딩한 뒤 JSON으로 파싱하면 됩니다 (Arduino 쪽 JSON 파싱 라이브러리 사용 권장 — 예: ArduinoJson).

6. 통신 흐름 예시 (REPLACE로 끝나는 한 사이클)

```
Jetson --ASSIGN_TARGET(target=cell_5)--> Mega
Mega: ■■ ■■ (■■ ■■, Jetson■ ■■■■ ■■)
Mega --REQUEST_VISION-----> Jetson
Jetson --VISION_RESULT(status=powdery_mildew)--> Mega
```

```
Mega: █ █ → REPLACE █, █ █ █  
(█ █ █ █ █ █ █ █ REQUEST_VISION █ █ █ █ █  
█ █ █ █ █ █ █ █ - █ █ █ █ █ █ █ █)  
Mega --REPORT_RESULT(target=cell_5, execute_task=REPLACE, completion=COMPLETE,  
success=true)--> Jetson  
Jetson: █ █ █ █ AWS █ █
```

7. Mega 펌웨어가 새로 구현해야 할 것 (체크리스트)

- ① ASSIGN_TARGET 수신 파싱 — "type":"ASSIGN_TARGET"이 오면 target 필드(셀 라벨)를 저장하고, 기존처럼 EEPROM 등으로 물리 좌표를 조회해 이동을 준비합니다.
- ② REQUEST_VISION 전송 타이밍 결정 — 셀에 도착해 비전 확인이 필요한 시점마다 이 메시지를 보냅니다. 응답(VISION_RESULT)이 올 때까지는 다음 동작으로 넘어가지 않고 대기해야 합니다.
- ③ VISION_RESULT 수신 후 판단 로직 실행 — 아래 8절의 판단 규칙표를 그대로 C++ 로직으로 구현합니다. (이 규칙은 기존 Jetson의 robot/planner.py에 있던 것과 동일하며, 이번에 그대로 이식하면 됩니다.)
- ④ 판단 결과에 따른 물리 동작 수행 — REPLACE/OBSERVE/SKIP 각각의 물리 시퀀스는 기존 펌웨어의 동작 로직을 그대로 사용하되, 이제 이 시퀀스를 시작하는 주체가 Jetson의 명령이 아니라 Mega 자신의 판단이라는 점만 다릅니다.
- ⑤ (REPLACE의 경우) 완료 후 검증이 필요하다면 REQUEST_VISION을 한 번 더 전송 — 응답으로 받은 status가 "healthy"인지로 성공/실패를 자체 판정합니다. Jetson은 이 판정에 관여하지 않습니다.
- ⑥ REPORT_RESULT 전송 — 사이클이 끝나면 target, execute_task, completion, success(해당 시)를 담아 1회 전송합니다. 이걸로 한 사이클이 마무리됩니다.
- ⑦ 오류/타임아웃 처리 (신규 설계 필요, 아래 9절 참고) — REQUEST_VISION을 보냈는데 일정 시간 응답이 없을 때 Mega가 어떻게 동작할지 정의가 필요합니다. 현재 Jetson 쪽에는 이 시나리오에 대한 확정된 규칙이 없습니다 — 함께 논의가 필요합니다.

8. REPLACE/OBSERVE/SKIP 판단 규칙 (그대로 이식할 스펙)

아래 표는 기존 Jetson의 robot/planner.py에 있던 규칙을 그대로 옮긴 것입니다.

VISION_RESULT로 받은 status 값을 기준으로 이 표대로 판단하면 됩니다. 표에 없는 값(또는 인식 불가)은 전부 SKIP으로 처리합니다(기본값).

status (VISION_RESULT로 수신)	execute_task (Mega가 결정)
healthy	OBSERVE
powdery_mildew	REPLACE
missing_plant	REPLACE
empty_cell / 그 외 인식 안 된 값	SKIP (기본값)

9. 알려진 미해결 사항 및 주의사항

① 세부 진행상황(progress) 스트리밍이 당분간 없어집니다. 기존에는 물리 동작의 세부 단계(이동 중 → 집는 중 → 내려놓는 중 → 완료 등)를 Jetson이 실시간으로 받아 AWS에 중계했지만, 새 프로토콜은 REPORT_RESULT (최종 결과 1회)만 있어 이 실시간 진행률 표시가 당분간 빠집니다. 필요성이 확인되면 Mega가 별도의 진행상황 메시지 타입을 추가로 보내는 방식을 함께 설계해야 합니다.

② REQUEST_VISION에 대한 응답이 오지 않는 경우의 대응이 아직 정의되지 않았습니다. Jetson의 비전 처리가 실패하거나 지연되는 경우, Mega가 얼마나 기다릴지, 재요청할지, 안전 정지할지 등의 규칙이 필요합니다. 이는 기존 구조에는 없던 새로운 실패 시나리오이므로, 함께 논의해 정해야 할 항목입니다.

③ 셀 지정(ASSIGN_TARGET) 자체는 당분간 그대로 유지됩니다. 즉, "어느 셀을 처리할지"는 여전히 Jetson(AWS)이 지정하며, Mega가 스스로 순회 순서를 정하는 방식으로는 바뀌지 않았습니다. 이 부분이 향후 바뀔 수도 있다는 점은 참고로 남겨둡니다.

10. 요약

Jetson은 이제 판단하지 않고 "셀 지정 + 비전 결과 제공 + 최종 보고 중계"만 담당합니다. Mega는 비전 결과를 요청해서 받아본 뒤 스스로 REPLACE/OBSERVE/SKIP을 판단하고 실행하며, 끝나면 결과를 1회 보고합니다. 새 메시지 4종(ASSIGN_TARGET / REQUEST_VISION / VISION_RESULT / REPORT_RESULT)의 정확한 필드와 예시는 5절, 판단 규칙은 8절을 참고해 구현해 주시면 됩니다. 궁금한 점이나 위 미해결 사항(9절)에 대한 의견은 언제든지 회신 부탁드립니다.